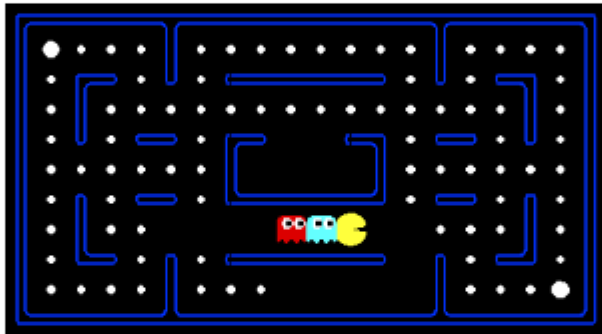


Projeto 2: Pesquisa Multiagente

Vencimento: quinta-feira, 20-11, 23h59



Pacman, agora com fantasmas.

Minimax, Expectimax,
Avaliação

ÍNDICE

- Introdução Bem-vindo ao Multi-
- Agent PacmanQ1 (4 pts): Reflex
- AgentQ2 (5 pts): Minimax
-
- Dicas e ObservaçõesQ3 (5
- pts): Poda Alfa-BetaQ4 (5
- pts): ExpectimaxQ5 (6 pts):
- Função de Avaliação

- Submissão

Introdução

Neste projeto, você projetará agentes para a versão clássica do Pacman, incluindo fantasmas. Ao longo do caminho, você implementará a pesquisa minimax e expectimax e tentará seu projeto de função de avaliação. A base de código não mudou muito em relação ao projeto anterior, mas comece com uma nova instalação, em vez de misturar arquivos do projeto 1. As no projeto 1, este projeto inclui um avaliador automático para você avaliar suas respostas em sua máquina. Isso pode ser executado em todas as perguntas com o comando:

```
autograder.py python
```

Ele pode ser executado para uma pergunta específica, como q2, por:

```
python autograder.py -q q2
```

Ele pode ser executado para um teste específico por comandos do formulário:

```
python autograder.py -t test_cases/q2/0-small-tree
```

Por padrão, o avaliador automático exibe gráficos com a opção -t, mas não com a opção -q. Você pode forçar gráficos usando o sinalizador --graphics ou não forçar nenhum gráfico usando o sinalizador --no-graphics. Consulte o tutorial do avaliador automático no Project 0 para obter mais informações sobre como usar o avaliador automático. O código para este projeto contém os seguintes arquivos, disponíveis como um arquivo zip.

Arquivos que você editará:

multiAgents.py

Onde todos os seus agentes de pesquisa multiagente residirão.

Arquivos que você pode querer ver:

pacman.py

O arquivo principal que executa os jogos do Pacman. Esse arquivo também descreve um tipo PacmanGameState, que você usará extensivamente neste projeto.

<code>game.py</code>	A lógica por trás de como o mundo Pacman funciona. Este arquivo descreve vários tipos de suporte, como AgentState, Agent, Direction e Grid.
<code>util.py</code>	Estruturas de dados úteis para implementar algoritmos de pesquisa. Você não precisa usá-los para este projeto, mas pode encontrar outras funções definidas aqui para serem úteis.
Arquivos de suporte que você pode ignorar:	
<code>graphicsDisplay.py</code>	Gráficos para Pacman
<code>graphicsUtils.py</code>	Suporte para gráficos Pacman
<code>textDisplay.py</code>	Gráficos ASCII para Pacman
<code>ghostAgents.py</code>	Agentes para controlar fantasmas
<code>keyboardAgents.py</code>	Interfaces de teclado para controlar o Pacman
<code>layout.py</code>	Código para ler arquivos de layout e armazenar seu conteúdo
<code>autograder.py</code>	Avaliador automático de projetos
<code>testParser.py</code>	Analisa arquivos de teste e solução do avaliador automático
<code>testClasses.py</code>	Classes gerais de teste de autoavaliação
<code>test_cases/</code>	Diretório contendo os casos de teste para cada pergunta
<code>multiagentTestClasses.py</code>	Classes de teste de avaliação automática específicas do Projeto 3

Arquivos para editar e enviar: Você preencherá partes de `multiAgents.py` durante a tarefa. Depois de concluir a tarefa, você enviará esses arquivos para o Gradescope (por exemplo, você pode fazer upload de todos os arquivos `.py` na pasta). Por favor, não altere os outros arquivos nesta distribuição.

Avaliação: Seu código será avaliado automaticamente para correção técnica. Por favor, não altere os nomes de nenhuma função ou classe fornecida dentro do código, ou você causará estragos no autoavaliador. No entanto, a exatidão de sua implementação – não os julgamentos do avaliador automático – será o juiz final de sua pontuação. Se necessário, revisaremos e avaliaremos as tarefas individualmente para garantir que você receba o devido crédito pelo seu trabalho. Desonestidade Acadêmica: Estaremos verificando seu código em relação a outros envios na classe para redundância lógica. Se você copiar o código de outra pessoa e enviá-lo com pequenas alterações, saberemos. Esses detectores de trapaça são muito difíceis de enganar, então, por favor, não tente. Confiamos em todos vocês para enviar apenas seu próprio trabalho; Por favor, não nos decepcione. Se o fizerem, procuraremos as consequências mais fortes de que dispomos.

Obtendo ajuda: Você não está sozinho! Se você estiver preso em algo, entre em contato com a equipe do curso para obter ajuda. O horário de expediente, a seção e o fórum de discussão estão disponíveis para seu apoio; por favor, use-os. Se você não puder comparecer ao nosso horário de expediente, avise-nos e agendaremos mais. Queremos que esses projetos sejam gratificantes e instrutivos, não frustrantes e desmoralizantes. Mas, não sabemos quando ou como ajudar, a menos que você pergunte. Discussão: Por favor, tome cuidado para não postar spoilers. Tópicos necessários para este projeto:

- Árvores de Jogo I: Vídeo 6, Nota 3.1-
- 3.2 Árvores de Jogo II: Vídeo 7, Nota 3.3-3.6

Bem-vindo ao Multi-Agent Pacman

Primeiro, jogue uma partida do Pacman clássico executando o seguinte comando:

```
pacman.py python
```

e usando as setas do teclado para mover. Agora, execute o ReflexAgent fornecido em multiAgents.py

```
python pacman.py -p ReflexAgent
```

Observe que ele funciona muito mal, mesmo em layouts simples:

```
python pacman.py -p ReflexAgent -l testClassic
```

Inspecione seu código (em multiAgents.py) e certifique-se de entender o que ele está fazendo.

Q1 (4 pts): Agente Reflexo

Melhore o ReflexAgent em multiAgents.py para jogar de forma respeitável. O código do agente reflex fornecido fornece alguns exemplos úteis de métodos que consultam o GameState para obter informações. Um agente de reflexo capaz terá que considerar os locais de alimentos e os locais fantasmas para ter um bom desempenho. Seu agente deve limpar de forma fácil e confiável o layout testClassic:

```
python pacman.py -p ReflexAgent -l testClassic
```

Experimente seu agente reflexo no meio padrãoLayout clássico com um fantasma ou dois (animação desligada para acelerar a exibição):

```
python pacman.py --frameTime 0 -p ReflexAgent -k 1
```

```
python pacman.py --frameTime 0 -p ReflexAgent -k 2
```

Como seu agente se sai? Provavelmente morrerá com 2 fantasmas no tabuleiro padrão, a menos que sua função de avaliação seja muito boa. Nota: Lembre-se de que newFood tem a função asList()Nota: Como recursos, tente o recíproco de valores importantes (como distância até a comida) em vez de apenas os valores em si. Observação: a função de avaliação que você está escrevendo está avaliando pares de estado-ação; Em partes posteriores do projeto, você avaliará os estados. Observação: pode ser útil exibir o conteúdo interno de vários objetos para depuração. Você pode fazer isso imprimindo as representações de string dos objetos. Por exemplo, você pode imprimir

```
newGhostStates com print(newGhostStates) .
```

Opções: Os fantasmas padrão são aleatórios; você também pode jogar por diversão com directionalghosts um pouco mais inteligentes usando -g DirectionalGhost . Se a aleatoriedade está impedindo você de dizer se seu agente está melhorando, você pode usar -f para executar com uma semente aleatória fixa (mesmas escolhas aleatórias em todos os jogos). Você também pode jogar vários jogos seguidos com -n . Desligue os gráficos com -q para executar muitos jogos rapidamente. Avaliação: Executaremos seu agente no layout openClassic 10 vezes. Você receberá 0 pontos se o seu agente expirar ou nunca ganhar. Você receberá 1 ponto se seu empresário vencer pelo menos 5 vezes, ou 2 pontos se seu empresário vencer todos os 10 jogos. Você receberá 1 ponto adicional se a pontuação média do seu agente for superior a 500 ou 2 pontos se for superior a 1000. Você pode testar seu agente nessas condições com

```
python autograder.py -q q1
```

Para executá-lo sem gráficos, use:

```
python autograder.py -q q1 --no-graphics
```

Não gaste muito tempo com essa questão, pois a carne do projeto está por vir.

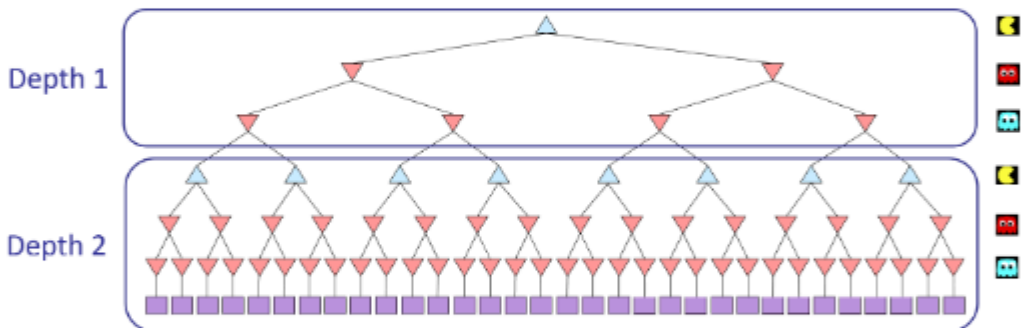
Q2 (5 pts): Minimax

Agora você escreverá um agente de pesquisa adversário no stub da classe `MinimaxAgent` fornecido em `multiAgents.py`. Seu agente minimax deve trabalhar com qualquer número de fantasmas, então você terá que escrever um algoritmo que seja um pouco mais geral do que o que você viu anteriormente na palestra. Em particular, sua árvore minimax terá várias camadas mínimas (uma para cada fantasma) para cada camada máxima.

Seu código também deve expandir a árvore do jogo para uma profundidade arbitrária. Marque as folhas da árvore `yourminimax` com o `self.evaluationFunction` fornecido, cujo padrão é `scoreEvaluationFunction`.

`MinimaxAgent` estende `MultiAgentSearchAgent`, que dá acesso a `self.depth` e `self.evaluationFunction`. Certifique-se de que seu código minimax faça referência a essas duas variáveis quando apropriado, pois essas variáveis são preenchidas em resposta às opções de linha de comando.

Importante: Uma única camada de busca é considerada um movimento do Pacman e todas as respostas dos fantasmas, então a busca de profundidade 2 envolverá o Pacman e cada fantasma se movendo duas vezes (veja o diagrama abaixo).



Classificação: Verificaremos seu código para determinar se ele explora o número correto de estados do jogo. Esta é a única maneira confiável de detectar alguns bugs muito sutis em implementações de minimax. Como resultado, o avaliador automático será muito exigente sobre quantas vezes você liga

`GameState.generateSuccessor`. Se você chamar de mais ou menos do que o necessário, o avaliador automático reclamará. Para testar e depurar seu código, execute

```
python autograder.py -q q2
```

Isso mostrará o que seu algoritmo faz em várias árvores pequenas, bem como em um jogo pacman. Para executá-lo sem gráficos, use:

```
python autograder.py -q q2 --no-graphics
```

Dicas e observações

- Implemente o algoritmo recursivamente usando funções auxiliares. A implementação correta do minimax fará com que o Pacman perca o jogo em alguns testes. Isso não é um problema: como é um comportamento correto, ele passará nos testes. A função de avaliação para o teste Pacman nesta parte já está escrita (`self.evaluationFunction`). Você não deve mudar essa função, mas reconhecer que agora estamos reavaliando estados em vez de ações, como estávamos para o agente reflexo. Os agentes de antecipação avaliam os estados futuros, enquanto os agentes reflexos avaliam as ações do estado atual. Os valores minimax do estado inicial no layout `minimaxClassic` são 9, 8, 7, -492 para profundidades 1, 2, 3 e 4, respectivamente. Observe que seu agente minimax geralmente vencerá (665/1000 jogos para nós), apesar da terrível previsão de profundidade 4 minimax.

```
python pacman.py -p MinimaxAgent -l minimaxClassic -a profundidade=4
```

- O pacman é sempre o agente 0 e os agentes se movem em ordem crescente de índice de agente. Todos os estados em minimax devem ser `GameStates`, passados para `getAction` ou gerados via `GameState.generateSuccessor` . Neste projeto, você não estará abstraindo para estados simplificados. Em tabuleiros maiores, como `openClassic` e `mediumClassic` (o padrão), você descobrirá que o Pacman é bom em não morrer, mas muito ruim em vencer. Ele costuma se debater sem progredir. Ele pode até se debater ao lado de um ponto sem comê-lo porque não sabe para onde iria depois de comer aquele ponto. Não se preocupe se você vir esse comportamento, a pergunta 5 resolverá todos esses problemas. Quando Pacman acredita que sua morte é inevitável, ele tentará terminar o jogo o mais rápido possível por causa da penalidade constante por viver. Às vezes, isso é a coisa errada a fazer com fantasmas aleatórios, mas os agentes minimax sempre assumem o pior:

```
python pacman.py -p MinimaxAgent -l trappedClassic -a profundidade = 3
```

Certifique-se de entender por que Pacman corre para o fantasma mais próximo neste caso.

Q3 (5 pts): Poda Alfa-Beta

Crie um novo agente que use a poda alfa-beta para explorar com mais eficiência a árvore minimax, em `AlphaBetaAgent` . Novamente, seu algoritmo será um pouco mais geral do que o pseudocódigo da palestra, portanto, parte do desafio é estender a lógica de poda alfa-beta apropriadamente para vários agentes minimizadores.

Você deve ver uma aceleração (talvez a profundidade 3 alfa-beta seja executada tão rápido quanto a profundidade 2 minimax). Idealmente, a profundidade 3 no `smallClassic` deve ser executada em apenas alguns segundos por movimento ou mais rápido.

```
python pacman.py -p AlphaBetaAgent -a profundidade=3 -l pequenoClássico
```

Os valores minimax de AlphaBetaAgent devem ser idênticos aos valores minimax de MinimaxAgent, embora as ações selecionadas possam variar devido a diferentes comportamentos de desempate. Novamente, os valores inimax do estado inicial no layout minimaxClassic são 9, 8, 7 e -492 para profundidades 1, 2,3 e 4, respectivamente.

Classificação: como verificamos seu código para determinar se ele explora o número correto de estados, é importante que você execute a remoção alfa-beta sem reordenar os filhos. Em outras palavras, os estados sucessores devem sempre ser processados na ordem devolvida por `GameState.getLegalActions` . Novamente, não chame `GameState.generateSuccessor` mais do que o necessário.

Você não deve podar a igualdade para corresponder ao conjunto de estados explorados por nosso avaliador automático. (De fato, alternativamente, mas incompatível com nosso avaliador automático, seria também permitir a poda na igualdade e invocar alfa-beta uma vez em cada filho do nó raiz, mas isso não corresponderá ao avaliador automático.)O pseudocódigo abaixo representa o algoritmo que você deve implementar para esta pergunta.

Alpha-Beta Implementation

α : MAX's best option on path to root
 β : MIN's best option on path to root

```
def max-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize v =  $-\infty$   
    for each successor of state:  
        v = max(v, value(successor,  $\alpha$ ,  $\beta$ ))  
        if v >  $\beta$  return v  
         $\alpha$  = max( $\alpha$ , v)  
    return v
```

```
def min-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize v =  $+\infty$   
    for each successor of state:  
        v = min(v, value(successor,  $\alpha$ ,  $\beta$ ))  
        if v <  $\alpha$  return v  
         $\beta$  = min( $\beta$ , v)  
    return v
```

Para testar e depurar seu código, execute

```
python autograder.py -q q3
```

Isso mostrará o que seu algoritmo faz em várias árvores pequenas, bem como em um jogo pacman. Para executá-lo sem gráficos, use:


```
python autograder.py -q q3 --no-graphics
```

A implementação correta da poda alfa-beta fará com que o Pacman perca alguns dos testes. Isso não é um problema: como é um comportamento correto, ele passará nos testes.

Q4 (5 pts): Expectimax

Minimax e alfa-beta são ótimos, mas ambos assumem que você está jogando contra um adversário que toma as melhores decisões. Como qualquer pessoa que já ganhou o jogo da velha pode dizer, nem sempre é esse o caso. Nesta pergunta, você implementará o ExpectimaxAgent, que é útil para modelar o comportamento probabilístico de agentes que podem fazer escolhas abaixo do ideal. Tal como acontece com a pesquisa e os problemas ainda a serem abordados nesta classe, a beleza desses algoritmos é sua aplicabilidade geral. Para agilizar seu próprio desenvolvimento, fornecemos alguns casos de teste baseados em árvores genéricas. Você pode depurar sua implementação em pequenas árvores do jogo usando o comando:

```
python autograder.py -q q4
```

A depuração nesses casos de teste pequenos e gerenciáveis é recomendada e ajudará você a encontrar bugs rapidamente. Uma vez que seu algoritmo esteja funcionando em pequenas árvores, você pode observar seu sucesso no Pacman. É claro que os fantasmas aleatórios não são agentes minimax ideais e, portanto, modelá-los com a pesquisa minimax pode não ser apropriado. O ExpectimaxAgent não assumirá mais o mínimo de todas as ações fantasmas, mas sim a expectativa de acordo com o modelo do seu agente de como os fantasmas agem. Para simplificar seu código, suponha que você só estará concorrendo contra um adversário que escolha entre seus

getLegalActions uniformemente ao acaso.

Para ver como o ExpectimaxAgent se comporta no Pacman, execute:

```
python pacman.py -p ExpectimaxAgent -l minimaxClassic -a profundidade=3
```

Agora você deve observar uma abordagem mais arrogante em ambientes fechados com fantasmas. Em particular, se Pacman perceber que pode estar preso, mas pode escapar para pegar mais alguns pedaços de comida, ele pelo menos tentará. Investigue os resultados destes dois cenários:

```
python pacman.py -p AlphaBetaAgent -l trappedClassic -a profundidade=3 -q -n 10
```

```
python pacman.py -p ExpectimaxAgent -l trappedClassic -a profundidade = 3 -q -n 10
```

Você deve descobrir que seu ExpectimaxAgent ganha cerca de metade das vezes, enquanto seu AlphaBetaAgent sempre perde. Certifique-se de entender por que o comportamento aqui difere do caso minimax. A implementação correta do expectimax fará com que o Pacman perca alguns dos testes. Isso não é um problema: como é um comportamento correto, ele passará nos testes.

Q5 (6 pts): Função de avaliação

Escreva uma função de avaliação melhor para o Pacman na função fornecida `betterEvaluationFunction`. A função de avaliação deve avaliar estados, em vez de ações como sua função de avaliação de agente reflexo fez. Com a pesquisa de profundidade 2, sua função de avaliação deve limpar o

pequenoLayout clássico com um fantasma aleatório mais da metade do tempo e ainda rodar a uma taxa razoável (para obter crédito total, Pacman deve ter uma média de cerca de 1000 pontos quando estiver ganhando).

Avaliação: o avaliador automático executará seu agente no layout `smallClassic` 10 vezes.

Atribuiremos pontos à sua função de avaliação da seguinte maneira:

Se você ganhar pelo menos uma vez sem cronometrar o autoavaliador, você recebe 1 ponto.

Qualquer agente que não satisfaça esses critérios receberá 0 pontos. +1 por vencer pelo menos

5 vezes, +2 por vencer todas as 10 vezes +1 por uma pontuação média de pelo menos 500, +2

por uma pontuação média de pelo menos 1000 (incluindo pontuações em jogos perdidos) +1 se

seus jogos demorarem em média menos de 30 segundos na máquina autoavaliadora, quando

executado com `--no-graphics`. Os pontos adicionais para pontuação média e tempo de

computação só serão concedidos se você ganhar pelo menos 5 vezes. Por favor, não copie

nenhum arquivo do Projeto 1, pois ele não passará no avaliador automático no

Gradescope. Você pode experimentar seu agente nessas condições com

.

```
python autograder.py -q q5
```

Para executá-lo sem gráficos, use:

```
python autograder.py -q q5 --no-graphics
```

Submissão

Para enviar seu projeto, carregue os arquivos Python que você editou. Por exemplo, use Gradescope's upload em todos os arquivos .py na pasta do projeto.
